

Federated Learning with Kalman Filter for Intrusion Detection in IoT Environment

Ziad Almesleh
College of Engineering
Qatar University

Ala Gouissem
College of Engineering and Technology
University of Doha for Science & Technology

Ridha Hamila
College of Engineering
Qatar University

Abstract—Enhancing the IoT (Internet of Things) network for reliability prompted a heightened focus on device security, given their diverse characteristics and sensitive data exchange. Federated Learning (FL) gained attention for its collaborative approach, sharing only local models, not data, among devices in IoT networks. Additionally, the non-i.i.d. data distribution complexity adds another layer to the network's challenges. In this article, we propose FedKF as a model approach for a federated learning algorithm with KF (Kalman Filter). This approach improves the performance of the IDS (Intrusion Detection System), especially for IoT data. In this model, each edge client trains the data locally to form a local model, which is then aggregated on a central server to create a global model. The FedKF aggregation algorithm employs a KF to predict and estimate the aggregation weight, where the prediction is based on the current measured weight of aggregated global models and the previous model weight. Furthermore, by selecting clients and allowing only selected devices to participate in the training process, the overall energy consumption can be reduced. Therefore, it's essential to balance energy savings with the performance of the federated learning model, ensuring that the model remains accurate and effective despite reduced participation.

The experimental results demonstrate the model's performance across different IoT datasets, compare the results with the average FL model, and verify the noticeable improvement in accuracy and communication loss. The model also shows the effect of client selection on the model's performance.

Keywords—Intrusion Detection System (IDS), IoT, Federated Learning, Non-IID, Kalman Filter.

I. INTRODUCTION

Internet of Things (IoT) devices are considered one of the most important components in the renaissance and revolution of communications. The design of modern communications networks is built on a substantial portion of IoT devices and the data exchanged between them. IoT devices span various fields, such as healthcare, industry, transportation, smart cities, and smart homes. The diversity of these devices and their data opens a wide door to emphasize the necessity of protecting these devices from vulnerabilities, especially when sharing data, which makes them more susceptible to attacks [1]. Furthermore, the main challenge in securing IoT devices lies in their limited resources; the constraints on CPU, memory, and energy storage capabilities make it difficult to run general-purpose security software, necessitating special-purpose security measures that are commensurate with the capabilities of IoT devices.

The intrusion detection system (IDS) is crucial for protecting data networks from attacks, detecting unusual behavior in network traffic. IDSs are divided into signature-based and anomaly-based types. Signature-based IDS, known for its simplicity, identifies known intruders through their unique footprints stored in a database. However, it struggles with zero-day attacks, which are not previously defined and can surge unexpectedly. Anomaly-based IDS, on the other hand, addresses this by using machine learning to establish a baseline of normal behavior to identify deviations, thus being more effective at spotting unknown attacks [2].

Artificial intelligence, through machine learning and deep learning algorithms, provides a method for developing intrusion detection systems. Machine learning involves statistical algorithms that learn to estimate and predict new data forms, classified into supervised or unsupervised learning based on data type [3]. Deep learning, mimicking the human brain, operates using artificial neural networks (ANN) with multiple layers, including input, output, and hidden layers. These layers contain neural networks that execute mathematical operations iteratively [4].

Federated learning is a machine learning training approach that operates in a decentralized manner, where edge devices participate in the training process instead of centralizing the entire training workload on the main server. In this setup, each edge device trains data locally and develops a local model. These local models are then shared with the main server to construct a global model, without sharing the underlying data. This global model is subsequently distributed to all edge devices within the network [5].

In this article, we apply the Kalman filter in conjunction with federated learning on the central server to enhance the performance of the global model. Local devices share their locally trained models, and then the central server aggregates these models to create an aggregated model. However, before sharing this aggregated model with edge devices, the central server employs a Kalman filter. This filter is based on the trained weights' current values and their previous values, updating the weights to form a refined global model. The main contributions of this paper are as follows:

- 1) We aim to improve the performance parameters in FL for non-iid data to implement a reliable, flexible, and powerful intrusion detection system for the IoT environment.

- 2) We propose an FedKF for implementing IDS in the IoT environment, which uses Kalman filter with federated learning to improve the performance of intrusion detection systems, especially for non-iid data.
- 3) We conducted experiments using different IoT datasets and compared them with existing approaches.
- 4) Finally, we proposed an energy-saving method based on the model presented, in which the federated learning process selects the most effective clients for the training process. This approach eliminates the need for all clients to participate in the training, yielding results that closely match those obtained when all clients are involved.

The remainder of this paper is organized as follows: Section II presents the related works on federated learning and the Kalman Filter. Section III depicts the proposed approach and the system architecture. Section IV describes the results. Finally, Section V concludes the paper.

II. RELATED WORKS

The Internet of Things (IoT) network has become a hot topic in research due to the widespread use of these devices and the need for them to communicate with each other. Consequently, the sharing of heterogeneous data and devices introduces new challenges and research opportunities. One of the primary challenges is securing IoT devices, a problem compounded by the nature of IoT data and the resource limitations of IoT devices. Federated learning is employed to develop a decentralized Intrusion Detection System (IDS) machine learning model. In this model, edge devices participate in the training process; each edge client is responsible for training local data, then shares this local model with a central server. The central server aggregates all local models into a global model and distributes it back to all edge clients. Figure 1 shows the federated learning steps.

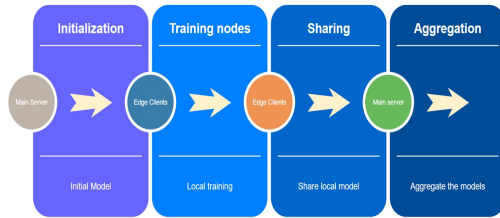


Fig. 1: Federated learning steps.

Furthermore, Chen et al. [6] proposed a federated learning algorithm based on perturbation defense to alleviate gradient leakage. In [7], Huang et al. introduced the Execution and Evaluation Dual Network Framework (EEFED) to improve the handling of imbalanced data and non-IID distribution by enhancing the local model to better reflect the global model, thus optimizing the local model based on enhanced backtracking parameters. In [8], Zuchao Ma et al. proposed Adaptive Collaborative Learning (ADCL), which creates a fingerprint of the trained model based on its topology, then classifies the models based on similarities to create a smart

model from these ensemble models, thereby improving the performance of each model.

In [9], Zhang et al. proposed a federated learning model based on transfer learning for each instance. The aggregation is implemented in two steps: employing ensemble model voting techniques and evaluating the weights. Moreover, Khoa et al. [10] proposed integrating federated learning with transfer learning to enhance performance. Transfer learning allows the model with rich data to transfer beneficial information to networks with low data. Meanwhile, Alcázar et al. [11] introduced a Fed+ model based on differential privacy techniques during training, while using a FedAvg aggregation algorithm for aggregation.

In [12], any device can request a cluster where the evaluation of clustering is processed by the Kalman filter, which enhances prediction accuracy. Also, in [13], Kalman filters are utilized to provide appropriate confidence values, such as noise levels and non-processing factors, etc., for local models. These filters then aggregate local models based on those confidence values. On the other hand, [14] implements the Kalman filter as a noise filter to minimize the impact of noise added to the model for the purpose of privacy preservation.

In [15], Xue Ma et al. used an extended Kalman filter (EKF) to adjust the local data of clients to network parameters, which were then sent as a global model. The central server used a sequential Kalman filter (SKF) as a fusion filter for the network parameters uploaded from the clients. In [16], Chen et al. implemented quantization for source coding, where Kalman filters were applied to determine the best quantization levels for each client and each label. In [17], Jiahao Du et al. implemented the forgetting Kalman filter (FKF) to fuse the client model and then used cubic exponential smoothing (CES) to predict the aggregation model.

III. PROPOSED APPROACH

The 'Proposed Approach' section of the report outlines each component of the proposed system: datasets, convolutional neural networks (CNNs), Kalman filters, federated learning, and the system architecture.

A. Non-IID dataset

In the context of federated learning and distributed data processing, distributing data efficiently among participating clients is crucial to ensure robust model training and generalization. This is particularly important in IoT environments, which are characterized by non-IID (non-Independently and Identically Distributed) data.

This paper introduces a methodology for dataset partitioning utilizing a predefined allocation matrix, M , to guide the distribution of data instances across multiple clients, specifically aiming to address the challenge of non-identically and independently distributed (non-IID) data. The dimensions of the M matrix are meticulously designed to accurately reflect the number of clients and unique classes within the dataset. The generation of the matrix M involves a controlled random process, governed by a variance parameter, d , which allows for

the customization of the data distribution to more closely simulate real-world scenarios where data is unevenly distributed. This procedure is detailed in Algorithm 1.

Algorithm1: Data Division Based on M Matrix.

Input: $num_clients$: Number of clients among which the data is to be distributed, $class_counts$: Dictionary containing counts of instances per class in table, $variance_parameter$: Standard deviation parameter d for generating matrix M , M Allocation matrix of size $num_clients \times num_classes$ dictating distribution of class instances per client.

Output: Datasets allocated to clients.

- 1: Extract unique class labels from the last column of table.
 - 2: For each class, generate a column in M using normal distribution with mean 0 and standard deviation d .
 - 3: Validate that M 's dimensions match $num_clients$ and the number of unique classes extracted from table.
 - 4: Create a $class_data$ that mapping each class label to its corresponding subset within table.
 - 5: For each client i and each class label j , allocate data instances as per $M[i, j]$, ensuring not to exceed the available instances for class j .
 - 6: Concatenate selected instances across all classes to form a comprehensive dataset.
-

B. Kalman filter

The Kalman filter is a state estimation method that integrates measurements and predictions to produce precise and effective estimates of the system's state. Thus, the Kalman filter employs a State Space Representation, where the state space consists of the current system state (the system's internal variables) and a measurement state (observed values). The prediction step is the first phase of estimation that estimates the future state value based on the input value and the previous state. The next step is the update step, which integrates measurements to refine the state estimate [18].

The Kalman filter calculation starts with the calculation of the Kalman gain, followed by the calculation of the current state estimation value, and finally, the calculation of the error value of the new estimation. This error value is used to calculate the Kalman gain for the next state. Equations (1), (2), and (3) show the calculations involved in the Kalman filter process.

$$KG = \frac{e_x}{e_x + e_m} \quad (1)$$

where KG is Kalman Gain, e_x is the estimation error, and e_m is the measurement error. then, the estimation for next state:

$$X_t = X_{t-1} + KG(M - X_{t-1}) \quad (2)$$

where, X_t is next state value, X_{t-1} is the previous state value, and M is the measurement value. finally, calculation the estimation error that used for KG calculation for next step.

$$e_{x_t} = \frac{(e_m)(e_{x_{t-1}})}{(e_m) + (e_{x_{t-1}})} \quad (3)$$

where e_{x_t} is the error in estimation at t state, $e_{x_{t-1}}$ is the error in estimation at $t-1$ state, and e_m is the error in measurement. Figure 2 show the diagram of Kalman filter.

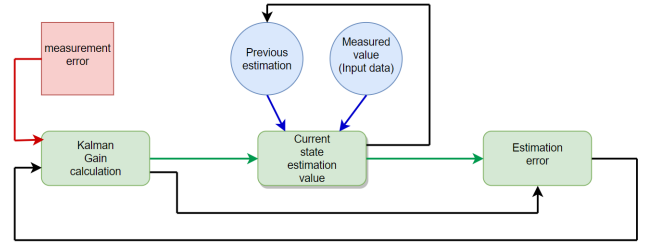


Fig. 2: Kalman filter diagram.

C. Federated learning

Federated learning is an innovative concept in machine learning that emphasizes user privacy while optimizing performance through a decentralized approach to model training. Federated learning, in contrast to conventional centralized learning, distributes the training process among several devices or servers, enabling them to cooperate and build a global model based on the shared local client models.

In general, the objective of federated learning (FL) is to find a model with parameters w that minimizes the cost function $F(w)$. This cost function is an average of the loss functions across all datasets, which can be expressed as follows:

$$F(w) = \sum_{i=1}^N \frac{D_i}{D} f_i(w) \quad (4)$$

where, N is total number of clients, D_i client dataset, D all dataset, and $f_i(w)$ is a loss function for client i .

In each training round t , the global model w_t is sent to a subset of devices. Each device i trains the model on its local data D_i , producing an updated model w_{t+1}^i by minimizing the local loss function L_i [34]:

$$w_{t+1}^i = w_t - \eta \nabla F_i(w_t, D_i) \quad (5)$$

where η is the learning rate, and ∇F_i is the gradient of the loss function L_i with respect to the model parameters w_t , calculated using the local dataset D_i . After training, the local model updates (or the gradients of the updates) $\Delta w_{t+1}^i = w_{t+1}^i - w_t$ are sent back to the central server. The server aggregates these updates to update the global model. A common aggregation method is Federated Averaging (FedAvg), where the global model is updated as follows:

$$w_{t+1} = w_t + \frac{1}{N} \sum_{i=1}^N \Delta w_{t+1}^i \quad (6)$$

This process ensures that the training data remains on the local devices, significantly enhancing data privacy and security. Moreover, Federated Learning is inherently robust to non-IID (independently and identically distributed) data, as the global model learns from a diverse dataset aggregated across many devices. figure 3 shows the federated learning architecture.

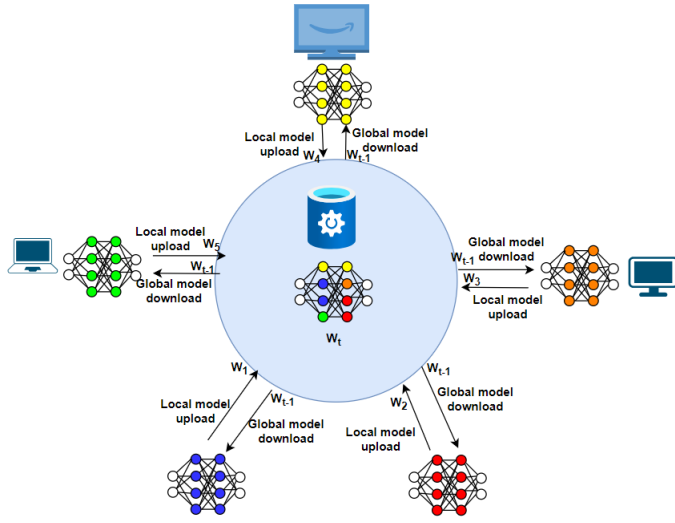


Fig. 3: Federated learning architecture.

D. System architecture

The proposed system model innovatively incorporates a Kalman filter into the federated learning framework to enhance the model updating process, aiming for improved convergence and accuracy across heterogeneously distributed datasets. The architecture of the model begins with a Convolutional Neural Network (CNN), which is specifically designed to handle a wide array of tasks. At the core of the model lies a federated learning framework, wherein a global model is trained across multiple iterations. Each iteration consists of local training sessions on clients' devices, followed by the aggregation of their updates to refine the global model.

The novel integration of a Kalman Filter into the federated learning process represents a significant advancement. The Kalman Filter, known for its efficacy in estimating the state of a linear dynamic system from a series of measurements, enhances the model updating process. Instead of directly averaging updates from local models, the system employs a Kalman Filter for the weights of each layer. This involves predicting the next state of the global model's weights and updating this prediction with the locally computed weights, derived from the diverse data of clients. After all selected clients have contributed their updates, the global model aggregates these filtered updates. This results in a refined set of weights that more accurately and stably reflects learning across the federation. This cycle repeats over multiple rounds, gradually improving the global model's performance. Figure 4 show the diagram of federated learning Kalman filter.

The Kalman filter, through a dynamic estimation process, enhances federated learning by selectively integrating updates from local models into the global model. This method allows choosing clients with the most beneficial data or updates at any time, eliminating the need for all clients to participate in each update cycle. This selective engagement improves the efficiency of the aggregation process and conserves energy. Given federated learning's demand on resources,

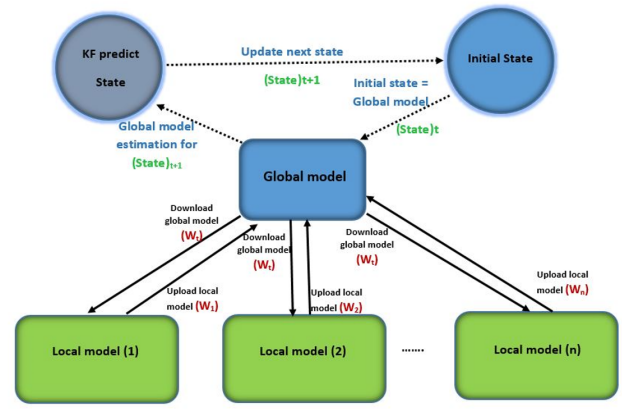


Fig. 4: Federated learning Kalman filter diagram.

Algorithm2: Federated learning using Kalman Filter as an aggregation algorithm.

Input: N number of clients with dataset D_i for each client, n number of communication round, Learning rate α .

Output: Trained Global Model G .

```

1: Initialize the global model  $G$ 
2: for each communication round  $r = 1$  to  $R$ :
3:   Select a subset of clients  $C_r$  for participation in round  $r$ .
4:   for each client  $c$  in  $C_r$ :
5:     Load client  $c$ 's data.
6:     Initialize a local model  $L_c$  with the same architecture as  $G$ 
       and set  $L_c$ 's weights to  $G$ 's weights.
7:     Train  $L_c$  on client  $c$ 's data.
8:     Extract  $L_c$ 's weights.
9:     for each weight layer  $w$  in  $G$ :
10:      Kalman filter update  $K$  with  $L_c$ 's corresponding weight
        layer.
11:      Set  $G$ 's weight layer  $nw$  to  $K$ 's state estimate.
12:    end for
13:  end for
14: end for

```

such as computation and communication, minimizing active client participation without affecting learning quality is a significant improvement. This approach optimizes energy use during training and updating, making the process more sustainable and practical, particularly in energy-constrained situations or where reducing consumption is crucial. This strategy represents a major step forward in balancing model performance with efficient resource use in distributed learning. Details of this system model are elaborated in Algorithm 2.

IV. RESULTS AND DISCUSSION

This chapter presents the findings of the study on the impact of the proposed federated learning with the Kalman filter algorithm on federated learning efficiency. The results are compared against the federated average algorithm to highlight the proposed algorithm's efficacy.

A. Performance Comparison

We start carry out experiments to compare the results of proposed model with FedAvg model, where the proposed model consists of a central server and 20 clients participated

in the training process, beginning from CIC-IoT-2023 dataset where this dataset is the most recent and real-time dataset.

Figure 5a shows the comparison of accuracy for proposed algorithm and FedAvg algorithm, where the figure shows that FedKF get clearly better performance than FedAvg where the accuracy for FedKF reach to 99.8% whereas in FedAvg the accuracy after 20 communication round 75.5%. On other hand for for the loss value for FedAvg as shown in figure 5b is 1.57 and no major change in the losses value as increase the communication rounds, whereas FedKF get better loss value starts from approximately 0.4 decrease to 0.07 loss value. moreover, we applied FedKF for CIC-IDS2018 dataset which these datasets are most popular for DDos attacks for IoT traffic to generalize the model. figure 6a shows that the accuracy for FedKF reach to 99.5% whereas in FedAvg the accuracy 98.0%, also the losses in figure 6b shows the loss using FedKF 0.038 whereas as for Fedavg is 1.08.

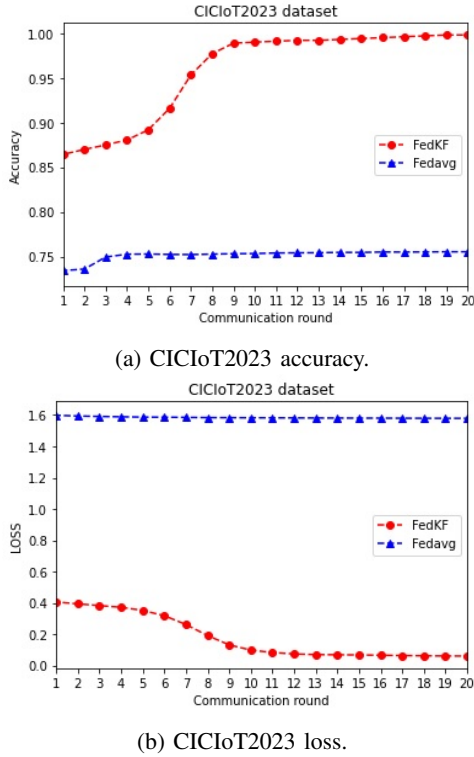


Fig. 5: Performance comparison between FedAvg and FedKF for CICIoT2023 dataset, using 20 edge clients and number of communication round =20.

B. Client Selection

Once the performance of the proposed approach has been verified, the next step is to investigate the energy savings for the model using the CIC-IoT2023 dataset. This begins by converting the CIC-IoT2023 dataset into a non-IID dataset based on the algorithm in Section III-A to more realistically simulate an IoT environment.

Client selection is handled in a unique and systematic way for each iteration of training. Specifically, in each of the 20

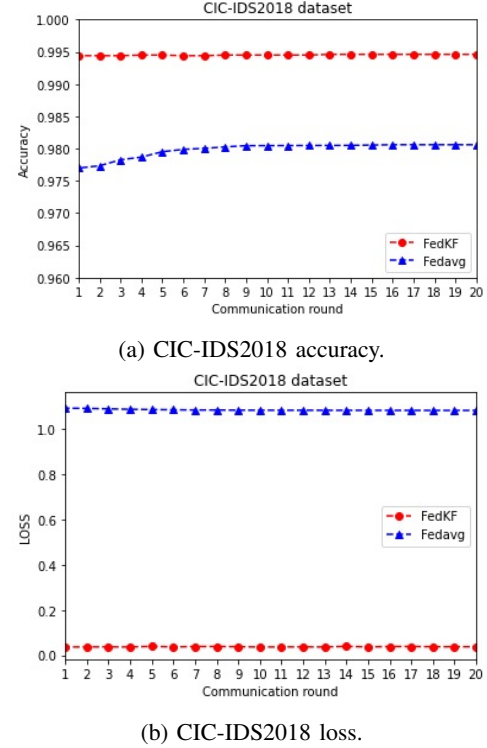


Fig. 6: Performance comparison between FedAvg and FedKF for CIC-IDS2018 dataset, using 20 edge clients and number of communication round =20.

iterations, one client is selected from 20 clients, ensuring that all clients except one participate in the training process. This approach serves to indirectly assess the impact of each client's data on the model's performance by observing changes in model accuracy and loss when specific clients are excluded. This method identifies the clients that have the least effect on the model's performance parameters. Excluding these clients reflects on energy consumption, thereby saving energy in the model.

Table I displays the impact on model performance for one unselected client. The table shows that Client 19 has the most significant effect on the model; excluding this client from the FL training process results in a decrease in system performance. On the other hand, client 4 has a lesser impact on the system, meaning that excluding this client does not significantly affect system performance.

While accuracy provides a high-level view of model performance, loss offers more granular insights into how well the model is fitting the data. This is particularly useful when comparing models or tuning hyperparameters to achieve optimal performance. Based on this, the accuracy and loss as indicated in Table ?? for FedKF with 20 clients on a non-IID CIC-IoT2023 dataset after 20 communication rounds are 93.29% and 0.327, respectively. Thus, excluding client 19 from the training process decreases performance to an accuracy rate of 86.10% and a loss value of 0.467. On the other hand, removing client 4 from the training process does not significantly change

TABLE I: Accuracy and Loss Metrics for Client Selections.

Client Selection	Accuracy%	Loss
Selected 20 Clients	93.29	0.327
Unselected clients: [1]	89.26	0.349
Unselected clients: [2]	87.95	0.406
Unselected clients: [3]	88.56	0.376
Unselected clients: [4]	92.59	0.329
Unselected clients: [5]	88.42	0.378
Unselected clients: [6]	89.02	0.384
Unselected clients: [7]	89.72	0.351
Unselected clients: [8]	88.46	0.392
Unselected clients: [9]	88.48	0.388
Unselected clients: [10]	90.37	0.384
Unselected clients: [11]	87.79	0.406
Unselected clients: [12]	88.22	0.365
Unselected clients: [13]	88.26	0.385
Unselected clients: [14]	88.49	0.389
Unselected clients: [15]	88.51	0.375
Unselected clients: [16]	88.63	0.382
Unselected clients: [17]	88.92	0.376
Unselected clients: [18]	88.56	0.374
Unselected clients: [19]	86.10	0.467
Unselected clients: [20]	88.17	0.386

performance, with the accuracy rate remaining at 92.59% and the loss value at 0.329, which is very close to the FL model that includes all clients.

In the context of Federated Learning (FL), excluding a specific client from the training process can lead to significant energy savings for both the excluded client and the central server responsible for model aggregation. By not participating in the training rounds, the client avoids the computational and communication overhead associated with training and transmitting updates, thereby conserving its resources. Concurrently, the central server benefits from a reduced load in processing and aggregating updates. This reduction in workload can lead to lower energy consumption and computational costs, as the server needs to integrate fewer updates into the global model.

V. CONCLUSION

In conclusion, this study presents a novel approach to federated learning by integrating a federated learning and Kalman Filter-based global model updates. Our method demonstrates significant improvements in learning efficiency and accuracy across distributed clients, with a strategic selection of clients for each communication round, thereby ensuring diversity and comprehensiveness in the learning process. The use of a Kalman Filter for updating the global model weights introduces a sophisticated method to incorporate new local updates while maintaining stability in the learning process. This approach not only enhances the model's performance but also contributes to energy savings by optimizing the number of clients engaged in each iteration, thus reducing unnecessary computations and communications. Our results, depicted through the accuracy and loss metrics over multiple communication rounds, affirm the potential of our method in handling federated learning tasks with improved efficiency and reduced energy consumption. This research opens new avenues for model update mechanisms within the federated

learning paradigm, potentially leading to more efficient and also energy-conservative machine learning models. In future research, we plan to delve into comparing our model against other federated learning models, particularly those surpassing the FedAvg benchmark, to ascertain their relative advantages in terms of efficiency, scalability, and energy consumption.

REFERENCES

- [1] D. C. Attota, V. Mothukuri, R. M. Parizi, and S. Pouriyeh, "An Ensemble Multi-View Federated Learning Intrusion Detection for IoT," *IEEE*, vol. 9, 2021.
- [2] A. Jamalipour and S. Murali, "A Taxonomy of Machine-Learning-Based Intrusion Detection Systems for the Internet of Things: A Survey," *IEEE Internet of Things Journal*, vol. 9, no. 12, June 15, 2022.
- [3] H. W. Oleiwi, D. N. Mhawi, and H. Al-Raweshidy, "MLTs-ADCNs: Machine Learning Techniques for Anomaly Detection in Communication Networks," *IEEE*, vol. 10, 2022.
- [4] M. E. Aminanto, R. S. H. Wicaksono, A. E. Aminanto, H. C. Tanuwidjaja, L. Yola, and K. Kim, "Multi-Class Intrusion Detection Using Two-Channel Color Mapping in IEEE 802.11 Wireless Network," *IEEE*, vol. 10, 2022.
- [5] Z. Chen, N. Lv, P. Liu, Y. Fang, K. Chen, W. Pan, "Intrusion Detection for Wireless Edge Networks Based on Federated Learning," *IEEE*, vol. 8, 2020.
- [6] J. Chen, Y. Zhao, Q. Li, X. Feng, K. Xu, "FedDef: Defense Against Gradient Leakage in Federated Learning-Based Network Intrusion Detection Systems," *IEEE Transactions on Information Forensics and Security*, vol. 18, 2023.
- [7] X. Huang, J. Liu, Y. Lai, B. Mao, H. Lyu, "EEFED: Personalized Federated Learning of Execution&Evaluation Dual Network for CPS Intrusion Detection," *IEEE Transactions on Information Forensics and Security*, vol. 18, 2023.
- [8] Z. Ma, L. Liu, W. Meng, X. Luo, L. Wang, W. Li, "ADCL: Toward an Adaptive Network Intrusion Detection System Using Collaborative Learning in IoT Networks," *IEEE Internet of Things Journal*, vol. 10, no. 14, July 15, 2023.
- [9] J. Zhang, C. Luo, M. Carpenter, G. Min, "Federated Learning for Distributed IIoT Intrusion Detection Using Transfer Approaches," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 7, July 2023.
- [10] T. V. Khoa, D. T. Hoang, N. L. Trung, C. T. Nguyen, T. T. T. Quynh, D. N. Nguyen, N. V. Ha, E. Dutkiewicz, "Deep Transfer Learning: A Novel Collaborative Learning Model for Cyberattack Detection Systems in IoT Networks," *IEEE Internet of Things Journal*, vol. 10, no. 10, May 15, 2023.
- [11] P. Ruzafa-Alcázar, P. Fernández-Saura, E. Mármol-Campos, A. González-Vidal, J. L. Hernández-Ramos, J. Bernal-Bernabe, and A. F. Skarmeta, "Intrusion Detection Based on Privacy-Preserving Federated Learning for the Industrial IoT," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, February 2023.
- [12] H. Kim, B. Kim, Y. Kim, C. You, H. Park, "K-FL: Kalman Filter-Based Clustering Federated Learning Method," *IEEE*, vol. 11, 2023.
- [13] K. Hu, J. Wu, L. Weng, Y. Zhang, F. Zheng, Z. Pang, M. Xia, "A novel federated learning approach based on the confidence of federated Kalman filters," *International Journal of Machine Learning and Cybernetics*, 2021, 12:3607–3627.
- [14] X. Yang and Z. Dong, "Kalman Filter-Based Differential Privacy Federated Learning Method," *Applied Science*, 2022, 12, 7787.
- [15] X. Ma, C. Wen, T. Wen, "An Asynchronous and Real-Time Update Paradigm of Federated Learning for Fault Diagnosis," *IEEE Transactions On Industrial Informatics*, vol. 17, no. 12, December 2021.
- [16] G. Chen, L. Yu, W. Luo, Y. Xu, T. Song, "Rate-Distortion Optimization for Adaptive Gradient Quantization in Federated Learning," *IEEE*, 2023.
- [17] J. Du, N. Qin, D. Huang, Y. Zhang, and X. Jia, "An Efficient Federated Learning Framework for Machinery Fault Diagnosis With Improved Model Aggregation and Local Model Training," *IEEE*, 2023.
- [18] P. S. Madhukar and L. B. Prasad, "State Estimation using Extended Kalman Filter and Unscented Kalman Filter," in *International Conference on Emerging Trends in Communication*, 2020.